
No API Key, No Problem: Enabling Agent-Centric Payments via Payment Channels

Liming Wang
Tsinghua University

Xiaoyu Fan
Tsinghua University

Wei Xu
Tsinghua University
Shanghai Qi Zhi Institute

Junda Liu
Tsinghua University

Abstract

Large language model (LLM)-based multi-agent systems (MAS) are rapidly emerging as a dominant paradigm for complex task automation. Despite their success, these systems rely on human-oriented API-key-based billing, which couples payment authority with long-lived, high-privilege credentials. This misaligned design introduces significant financial risks, onboarding overhead, and upfront funding costs. A naive alternative is to enable agents to transact directly using blockchain-based payments, eliminating the need for centralized billing and credit-based access. However, systematic profiling of real MAS execution reveals that the agent payment workload is characterized by high-frequency, low-value transactions, near-linear cost accumulation, and heterogeneous per-agent cost distribution, making blockchain-based payments impractical in both cost and latency. Motivated by these observations, we propose *PACT (Payment System for Agent-Centric Transactions)*, a layered payment system based on payment channels tailored to MAS workloads. To simultaneously handle heterogeneous MAS frameworks, ensure secure payment execution without exposing credentials, and support efficient, robust settlement for high-frequency workloads, PACT abstracts diverse MAS frameworks into a unified interface, introduces a cost-aware middleware for fine-grained usage tracking, and employs a supervisor to coordinate MAS execution with payment-channel operations while isolating credentials. Comprehensive evaluation across multiple MAS frameworks with diverse foundation models demonstrates that PACT effectively avoids the substantial overhead of on-chain settlement, preserving task success rate yet introducing negligible latency and cost overhead compared to API-key-based systems.

1 Introduction

Large language model (LLM)-based multi-agent systems (MAS) have rapidly evolved from research prototypes into production-scale infrastructure, showing capabilities across multiple domains including software engineering [1, 2], embodied planning [3, 4], scientific discovery [5, 6], and strategic reasoning [7]. These advances signal a broader shift: multi-agent systems are emerging as a new computational paradigm in which autonomous agents, rather than humans, act as the primary units of interaction, coordination, and execution.

This emerging paradigm requires not only intelligence and coordination, but also a shift toward *agent-centric* system design. Many existing infrastructures, however, remain fundamentally human-centric, leading to a mismatch when applied to autonomous agents. In particular, the economic layer of the LLM ecosystem relies on API-key-based billing, where providers like OpenAI, Google, Microsoft, etc., authenticate users through long-lived credentials tied to centralized, credit-based billing accounts. This human-centric design introduces several limitations for agents: (1) it couples payment authority with private and high-privilege credentials, resulting in security risks; (2) it requires provider-specific onboarding, introducing significant overhead for dynamic, multi-provider

agent workflows; and (3) it supports only coarse-grained, post hoc billing, limiting real-time and fine-grained control (see §2.3 for details).

A natural direction towards agent-centric economics is blockchain-based settlement, which provides a trust-minimized and universally verifiable payment layer, eliminating reliance on centralized billing and enabling direct, programmatic transactions between agents. The key question is whether such a settlement layer can match the workload of real MAS execution. We therefore profile MAS execution across representative frameworks and models (§3), revealing three key patterns: (1) transactions are high-frequency and low-value, (2) cumulative cost grows near-linearly over time, and (3) cost is moderately dispersed across agents within a task. These patterns rule out naïve per-call on-chain settlement due to prohibitive cost and latency, and instead motivate *Payment Channel Networks (PCNs)* [8, 9], which amortize many off-chain micropayments into a small number of on-chain transactions at near-zero marginal cost. In PCNs, users lock funds into payment channels and perform a sequence of off-chain, cryptographically signed balance updates with multiple LLM/service providers, allowing rapid, low-value payments without per-transaction blockchain overhead or centralized billing. Therefore, PCNs are well-suited for MAS workloads with repeated provider interactions and continuous fine-grained payments.

However, applying such mechanisms in MAS is non-trivial. While PCNs provide a suitable settlement primitive, they do not directly integrate with MAS frameworks. First, heterogeneous frameworks differ in topology, scheduling, and state representation, making it difficult to unify payment processing. Second, payments must be executed during agent runtime while keeping credentials secure; directly allowing agents to pay providers would couple payment authority with execution and significantly increase the attack surface. Third, payment updates must balance responsiveness and efficiency: frequent updates improve control but incur overhead, whereas infrequent updates amortize cost at the expense of delayed settlement and increased risk. Mapping continuous execution into discrete payment-channel updates, therefore, presents a fundamental system challenge. To address this, we propose *PACT (Payment System for Agent-Centric Transactions)*, a layered architecture that decouples agent execution from economic logic (§4). PACT treats the MAS as an unmodified component, ensuring compatibility across diverse frameworks, and introduces a cost-aware middleware that intercepts LLM calls to construct a unified, fine-grained cost representation. A dedicated supervisor then coordinates MAS execution with payment-channel operations through dual skills: MAS skills for invoking the underlying system and PCN skills for handling settlement, including channel management, balance updates, and credential handling.

We validate PACT across three representative MAS frameworks spanning distinct coordination topologies: MARBLE [10], ChatDev [1], and MacNet [11], each evaluated with three backbone models under diverse settings, including varying budget policies and multiple LLM providers. Compared to naïve on-chain settlement, which incurs prohibitive overhead in cost, latency, and reliability, PACT introduces negligible overhead across all dimensions, enabling efficient high-frequency micropayments without degrading system performance. Furthermore, even compared to API-key-based systems, PACT preserves task success rate while introducing negligible runtime and cost overhead, demonstrating that decoupling economic logic does not interfere with agent execution.

In summary, the contributions include:

- (1) Through systematic profiling of real MAS workloads, we reveal key properties—high-frequency, low-value transactions, near-linear cost accumulation, and heterogeneous per-agent cost distribution—which render both API-key-based billing and naïve on-chain settlement unsuitable, and motivate amortized off-chain settlement via payment channels.
- (2) We propose PACT, a system that decouples agent execution from economic logic, enabling secure, efficient, and interoperable payment across heterogeneous MAS frameworks.
- (3) We evaluate PACT across multiple MAS frameworks and backbone models, showing that it preserves task success rate with negligible overhead compared to API-key-based systems, while avoiding the prohibitive latency and cost of on-chain settlement.

2 Background and Related Works

2.1 Multi-Agent Systems

Applications of multi-agent systems. LLM-based multi-agent systems (MAS) have rapidly expanded across software engineering [1, 11–13], mathematical reasoning [14], scientific discovery [5, 6], embodied planning [3, 4], and strategic reasoning [7]. General-purpose orchestration frameworks and interoperability protocols further establish MAS as an infrastructure primitive [15–20].

Diversity in MAS coordination structures. MAS frameworks exhibit diverse coordination structures, ranging from static workflows with predefined roles, such as ChatDev [1] and MetaGPT [12], to dynamic topologies that adapt during execution, such as MacNet [11], Flow [21], AFlow [14], and MARBLE [10]. This diversity leads to substantial heterogeneity in topology, scheduling, and communication, motivating framework-agnostic payment integration.

2.2 Blockchain and Payment Channels

Blockchain as a payment substrate. Blockchain systems provide a trust-minimized and globally verifiable payment infrastructure, enabling mutually untrusted parties to transact without relying on centralized intermediaries [22]. This property makes blockchain a natural foundation for agent-centric payment, where autonomous agents interact across organizational boundaries and require direct, programmatic settlement without human-managed billing or credit-based access.

Payment channels. On-chain settlement requires every transaction to be broadcast, validated by all nodes, and recorded permanently, incurring per-transaction gas fees and finality delays (seconds to minutes depending on the chain). Payment channels are designed to avoid this: two parties lock collateral with a single on-chain transaction, then exchange arbitrarily many signed balance updates off-chain—with zero gas cost, instant finality, and no involvement of the global consensus—and settle the final state with one closing transaction [23]. The on-chain cost is therefore constant (2 transactions: open and close) regardless of how many off-chain updates occur.

2.3 Agent Payments

API-key billing model. MAS frameworks typically inherit API-key-based billing as their interface to model services, where agents are authenticated and charged through long-lived bearer tokens tied to human accounts. This model mismatches MAS workloads: keys act as uncapped credit lines without per-agent accountability, are provider-specific, lack real-time budget enforcement, and expose a broad attack surface when shared with agents.

Agent-centric payment designs. Existing systems address parts of the agent-payment problem but do not provide a unified solution for MAS workloads. Payment protocols such as x402 [24] pair API requests with on-chain payments, enabling direct access but incurring per-call settlement overhead. Blockchain infrastructures such as Nevermined [25] and SingularityNET [26] provide relayed payments or AI marketplaces, but do not integrate fine-grained payments into the MAS execution loop. Agent interaction and observability systems such as Google A2A [20], Helicone [27], LangSmith [28], and Lunary [29] standardize coordination or monitor costs post hoc but lack real-time payment execution and control.

3 Payment Workload Characterization

Setup. We instrument three MAS frameworks—MARBLE [10] (star), ChatDev [1] (chain), and MacNet [11] (DAG)—each paired with three self-hosted models (DeepSeek-V3.2, GPT-OSS-120B, Qwen3-32B) on their canonical benchmarks (MultiAgentBench Coding [10]; SRDD [11]), yielding 6,589 LLM call records across 270 tasks (details in §5.1).

Payment patterns. Across the 6,589 LLM call records, we identify three key patterns:

(1) *MAS payment workloads are characterized by high-frequency, low-value transactions.* Figure 1 plots the joint distribution of per-call cost and inter-call interval. LLM calls are extremely cheap yet frequent: the median per-call cost is \$0.0012, with the 95th percentile below \$0.0041, far be-

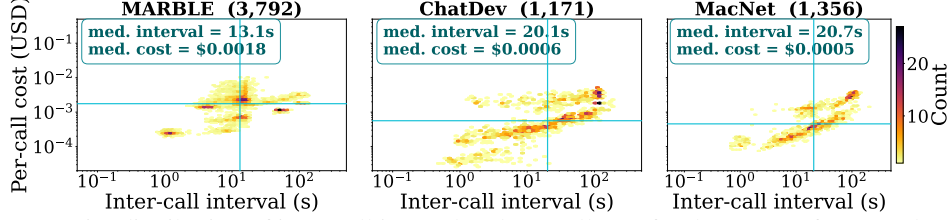


Figure 1: Joint distribution of inter-call interval and per-call cost for three MAS frameworks (6,589 calls in total). Color intensity indicates density; cyan crosshairs mark per-framework medians.

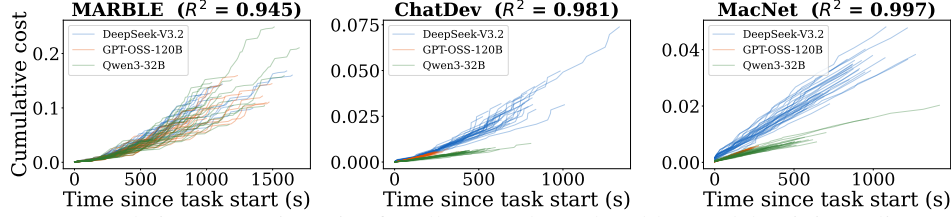


Figure 2: Cumulative cost trajectories for all 270 tasks, colored by model. Fitting a linear model to cumulative cost vs. wall-clock time yields $R^2 \geq 0.945$ for every framework ($R^2=1$ is perfect linearity), confirming near-constant spending rates.

low representative gas fees such as \$0.34 on Ethereum L1 [30]. Median inter-call intervals range from 13.1 s (MARBLE) to 20.7 s (MacNet), with bursts below 2.4 s, showing that MAS workloads generate continuous, bursty streams of sub-cent payments.

(2) *The cumulative cost of a single MAS task grows approximately linearly over time.* Figure 2 plots cumulative cost against wall-clock time across tasks, frameworks, and models. Linear regression yields $R^2 \geq 0.945$ across all frameworks, indicating near-constant spending rates during execution. Thus, MAS workloads require continuous, low-overhead settlement.

(3) *Inter-agent cost distributions show moderate but non-negligible variation across MAS frameworks.* Figure 3 plots per-agent cost distributions across tasks. Costs are neither uniform across agents nor dominated by a single agent: Gini coefficients range from 0.26 (MARBLE) to 0.54 (ChatDev). This motivates fine-grained, per-agent cost attribution and flexible control rather than uniform or task-level billing.

Naive on-chain settlement is infeasible. Directly settling each LLM call as an individual blockchain transaction is structurally incompatible with MAS workloads across three dimensions (Figure 4). First, the cost gap is prohibitive: on Ethereum L1 [30], the median transaction fee (\$0.34) exceeds the median LLM call cost by $283\times$; even on the cheapest L2 networks [31], the gap remains substantial (Base \$0.01, $8.3\times$; Arbitrum \$0.20, $167\times$). Second, latency is misaligned: blockchain finality ranges from ~ 13 s (Solana) to ~ 13 min (Ethereum L1), whereas MAS burst intervals can be as short as 2.4 s. Third, reliability degrades rapidly: at Ethereum’s $\sim 1\%$ failure rate, a 30-call task completes all settlements with only $(0.99)^{30} = 74\%$ probability; Solana’s 6–73% rate makes completion near-impossible. A full comparison across eight chains is provided in Appendix Table 4.

4 PACT: Payment System for Agent-Centric Transactions

4.1 Motivations

The payment patterns of §3 and the architectural diversity of existing MAS frameworks jointly yield two design insights:

(II) *Continually amortized sub-cent micropayments:* the first two patterns in §3 show that MAS calls are high-frequency and ultra-low-value with near-linear cost accumulation, so micropayments arrive at a roughly constant rate. The system must therefore *continuously* amortize payments into a small number of settlement events rather than batching at task boundaries.

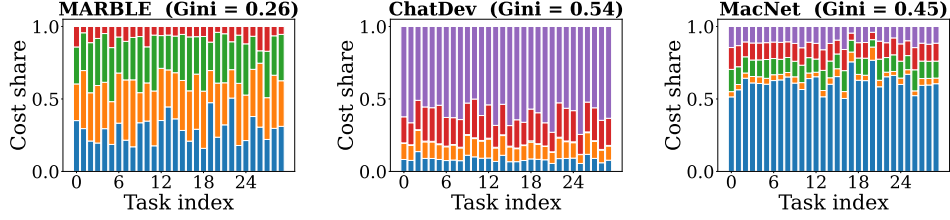


Figure 3: Inter-agent cost share within individual tasks. Gini coefficients range from 0.26 (MARBLE) to 0.54 (ChatDev), indicating moderate but non-negligible cost dispersion across agents.

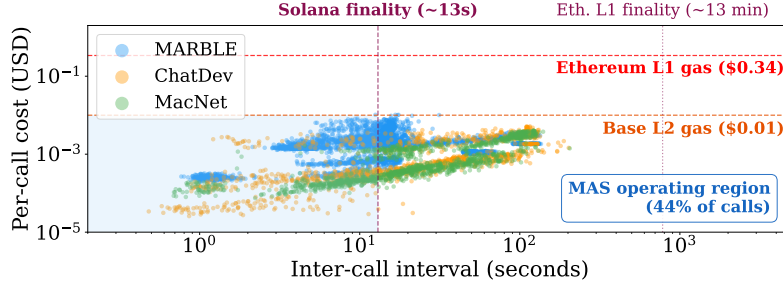


Figure 4: MAS call characteristics vs. blockchain settlement constraints (6,589 calls). Dashed lines mark gas fees (horizontal) and finality times (vertical); the shaded region contains 44% of all calls, where per-call on-chain settlement is infeasible on every major chain.

(12) *Framework-agnostic payment logic*: the third pattern shows that cost allocation varies substantially across frameworks (Gini 0.26–0.54), and their architectural heterogeneity (§2.1) rules out embedding payment logic inside any single framework. The payment layer must therefore treat the MAS as a black box, interposing only at the LLM call boundary.

Payment channels as the amortization primitive. *Payment channels* (§2) provide exactly the amortization required by II: each channel settles an arbitrary number of off-chain micropayments between just two on-chain transactions, yielding near-zero marginal cost per update. Because all LLM calls are routed through a small set of P model providers (typically single digits), the system maintains only $O(P)$ channels, making setup and teardown overhead negligible relative to the hundreds of micropayments each channel amortizes. Building on this primitive, we propose PACT (Payment system for Agent-Centric Transactions), a three-layer architecture (Figure 5).

4.2 PACT Architecture

Overview. PACT is a layered architecture that decouples agent execution from economic logic, enabling efficient, secure, and framework-agnostic payment for multi-agent systems (Figure 5). Layer 1 runs the MAS framework unmodified, preserving all existing agent workflows and behavior. Layer 2 introduces a cost-aware middleware that intercepts LLM calls, extracts per-agent usage, and constructs a unified representation of execution state. Layer 3 is a supervisor that coordinates MAS execution and payment-channel operations, serving as the sole holder of payment authority.

This layered design provides three key benefits. First, it isolates economic logic from heterogeneous MAS implementations, enabling compatibility across diverse frameworks without modification. Second, it separates payment triggering from payment authority, ensuring that credentials remain protected while allowing payments to be driven by agent execution. Third, it enables flexible control over settlement behavior, allowing the system to balance efficiency, robustness, and cost through configurable policies.

(1) Unified abstraction middleware. At startup, the middleware wraps each framework’s LLM gateway, intercepting all outbound LLM calls without modifying the underlying implementation, and augments each call with (i) a *pause gate* that can block or resume agents while preserving their working state, and (ii) fine-grained *cost recording* into a per-agent ledger.

Because different MAS frameworks expose incompatible state representations, PACT further introduces a *plugin abstraction*: each framework-specific adapter implements a common interface

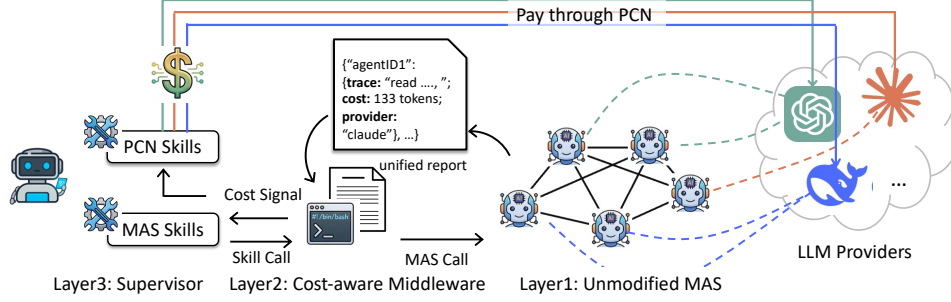


Figure 5: PACT system overview. *Service path*: LLM calls flow through the middleware (Layer 2) to providers. *Payment path*: accumulated costs are settled off-chain via per-provider payment channels with $O(P)$ on-chain transactions.

Table 1: Plugin adapter implementation: how each framework-extracted field in Algorithm 1 is realized. Other middleware fields (cost ledger, agent status) are the same and therefore omitted.

Framework	Topology	GETPROGRESS	GETHISTORY	GETMEMORY
MARBLE	star/chain/tree	iter i/N + planner summary	planner’s sub-task list	working_memory
ChatDev	SOP chain	active phase	per-role phase participation	ChatEnv decisions
MacNet	DAG	graph type + completed nodes	solution file list + line counts	node suggestions

(Table 1), and a report-construction procedure (Algorithm 1, Appendix) fuses *middleware-produced* fields (cost, status) with *framework-extracted* fields (progress, history, memory) into a structured *unified report*.

(2) Credential-isolated supervisor. The supervisor is an LLM-based agent whose tool set is partitioned into two disjoint skills—*MAS skill* and *state-channel skill*—providing two layers of credential isolation. First, the partition decouples payment *authority* from *triggering*: Layer 1 agents interact only with the LLM gateway intercepted by Layer 2 and have no execution path to the state-channel skill; cost events trigger settlement, but only the supervisor holds the authority to sign. Second, encapsulating state-channel operations behind a tool-use interface confines the keystore to a sign-or-reject API: the supervisor can *invoke* signing but never *read* raw key material, so even a compromised prompt cannot exfiltrate private keys.

(3) Two-level settlement scheduling. The per-agent ledger maintained by the middleware produces two threshold signals that bridge Layer 2 and Layer 3. A **unit signal** fires whenever unpaid cost accumulates to a configurable quantum u (e.g., \$0.005): the supervisor automatically issues an off-chain state-channel update—no agents are paused, and no LLM invocation occurs—enforcing *pay-on-use* at near-zero system cost. A **balance signal** fires when total task cost reaches a budget threshold $B \gg u$: all agents are paused, and the supervisor ingests the full unified report (Algorithm 1), then executes its budget policy—approving additional funds and resuming execution, or terminating the task (§5.3). The two levels are complementary: a small u maximizes provider trust via frequent, scripted payments, while a large B amortizes the costly supervisor reasoning across many calls and enables informed budget decisions. PACT exposes both as user-configurable parameters; we evaluate the effect of B on task quality in §5.3.

5 Experiments

We evaluate PACT along three dimensions: whether the payment layer introduces overhead on task quality or runtime, how budget-enforcement policies affect quality under cost constraints, and whether scaling to multiple concurrent payment channels introduces additional overhead.

5.1 Setup

MAS framework. We evaluate three MAS frameworks spanning distinct coordination topologies: MARBLE [10] (centralized star), ChatDev [1] (sequential chain), and MacNet [11] (DAG). MAR-

BLE is evaluated on MultiAgentBench Coding (30 tasks); ChatDev and MacNet on SRDD [11] (30 tasks each).

Metrics. For each configuration we collect task rating R , wall-clock time T , total cost C , and cumulative Celer payment time T_{celer} . Task rating is framework-specific: *Coding score* (MARBLE) is the mean of four LLM-as-judge dimensions (instruction following, executability, consistency, code quality), rescaled to 0–100; *Task score* (ChatDev, MacNet) is the mean of completeness, executability, and consistency sub-scores, rescaled to 0–100.

Backbone LLMs. Three models are used as the backbone for all MAS frameworks: DeepSeek-V3.2 [32], GPT-OSS-120B [33], and Qwen3-32B [34]. All LLM calls are routed through a LiteLLM proxy [35] to these models.

Baselines. We compare PACT against two baselines: (1) *API-Key* baseline, which is the original MAS execution setting, where agents invoke LLM providers through conventional API-key-based billing, and no additional payment layer is introduced. This baseline represents the performance target for PACT: an ideal agent-centric payment system should preserve task rating, runtime, and monetary cost relative to existing API-key-based execution. (2) *Blockchain* baseline, which keeps the same PACT structure but replaces payment-channel settlement with direct on-chain settlement. Whenever PACT would issue a payment-channel update at a unit signal, the blockchain baseline instead submits an on-chain transaction to settle the corresponding payment. Because blockchain systems exhibit varied performance characteristics, we use three representative chains: Ethereum L1 [36] (high fee, long finality), Base [37] (medium fee, short finality), and Solana [38] (low fee, short finality but high failure rate).

Implementation. We instantiate the state-channel layer on Celer Network [9], the most mature generalized payment channel implementation on Ethereum, supporting a programmatic CLI that enables fully automated channel management without manual wallet interaction. All Celer contracts are deployed on a local Geth testnet to ensure reproducible settlement conditions independent of mainnet gas-price variability.

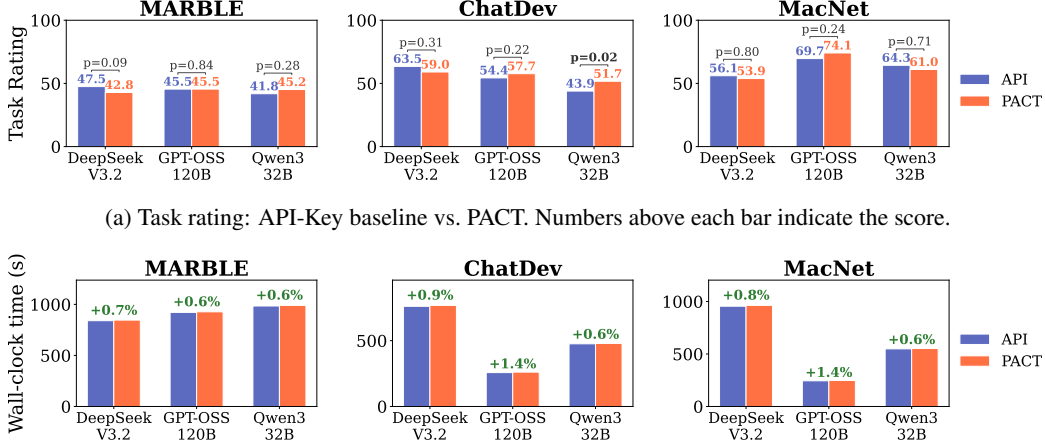
5.2 Main Results

We evaluate PACT against the two baselines across $9 = 3 \times 3$ MAS frameworks and backbone model pairs. Specifically, to isolate payment overhead from budget enforcement, we set the balance $B = \$999$ (effectively unlimited) and unit signal $u = \$0.005$ (indicating state-channel update frequency $\$4.2$). We have the following main observations:

PACT preserves task rating. Across all framework–model pairs, PACT has a negligible impact on task rating compared to the API-key baseline. As shown in Figure 6a, the task rating difference between PACT and API-key billing is small, with changes within ± 8 points on a 0–100 scale and no consistent pattern of degradation, confirming that the payment layer does not interfere with MAS execution or outcomes. A Wilcoxon signed-rank test further confirms that the rating differences are not statistically significant in 8 out of 9 configurations ($p > 0.05$), with only one configuration showing a marginally significant difference (ChatDev with Qwen3-32B, $p = 0.02$), which remains non-significant after Bonferroni correction of 9 group comparisons (adjusted $p = 0.18$). In contrast, the Solana [38] blockchain baseline substantially reduces task rating, as shown in Table 2, because each LLM call with on-chain settlement introduces a 6–73% transaction failure rate, causing tasks to fail midway with high probability and resulting in zero task score. This demonstrates that PACT decouples payment from task execution without degrading MAS outcomes.

PACT introduces negligible runtime overhead. PACT also preserves end-to-end execution time relative to API-key billing. Figure 6b shows that live payment-channel settlement adds only 0.6%–1.4% runtime overhead across framework–model pairs. By contrast, the blockchain baseline incurs substantial delay because every unit signal must wait for on-chain transaction submission and confirmation, introducing latency on the order of seconds or longer per payment. Thus, PACT supports high-frequency agent payments without disrupting the temporal dynamics of MAS execution.

PACT adds near-zero monetary overhead. Table 2 compares the additional settlement cost of PACT and the blockchain baseline under the same unit threshold. Because PACT amortizes many off-chain updates through payment channels, the marginal cost of each update is effectively zero; the only on-chain cost comes from opening and closing channels per framework. In contrast, the



(b) Wall-clock time: T_{API} (MAS execution time using API-Key) vs. T_{PACT} (total time with PACT). Percentages show the relative increase, all below 1.5%.

Figure 6: Payment overhead of PACT across three frameworks and three models.

Table 2: Per-unit settlement cost comparison. *Finality* refers to soft finality, i.e. the time from transaction submission to the first confirmation the application can act on. For the blockchain baseline, the average fee per unit equals the transaction fee. For PACT, the only on-chain cost is opening and closing the payment channel (two L2 transactions); the *avg fee per unit* is therefore $(2 \times f_{L2}) / N_{unit}$, where f_{L2} is the per-transaction L2 fee and N_{unit} is the total number of unit signals fired in a 30-task session. *Fail rate* is the fraction of submitted transactions that revert or are dropped.

Chain	Avg fee per unit (USD)			Finality	Fail rate
	MARBLE	ChatDev	MacNet		
Ethereum L1	\$0.34	\$0.34	\$0.34	~13 min	<1%
Base (L2)	\$0.01	\$0.01	\$0.01	~1 s	<1%
Solana	\$0.00025	\$0.00025	\$0.00025	~13 s	6–73%
PACT	\$0.00004	\$0.00024	\$0.00023	<50 ms	~0%

blockchain baseline pays a transaction fee for every unit-level settlement, causing payment overhead to exceed the underlying MAS execution cost when the unit threshold is small, resulting in a $35.7 \times - 250 \times$ increase in transaction cost compared to PACT across three MAS frameworks. These results confirm that payment channels are necessary for economically viable high-frequency micro-payments in MAS.

5.3 Budget Enforcement Policy

We evaluate how budget enforcement affects MAS performance when payment is coupled with execution-time control. We fix the model to DeepSeek-V3.2 and set the per-task budget to $B_i = \alpha \cdot C_{base,i}$, where $C_{base,i}$ is the average PACT cost for the corresponding framework and $\alpha \in \{0.75, 1.00\}$. We compare two policies: P1 (Hard Stop) terminates execution once the budget is reached; P2 (Active Approve) invokes the supervisor at the budget boundary, reads the unified report, and either approves an incremental budget extension if the task is likely to succeed, or terminates otherwise. Figure 7 plots the task rating of P1 with $\alpha = \{0.75, 1.0\}$, P2 with $\alpha = 0.75$ and the original PACT with an unlimited budget. We have the following conclusions:

(1) Hard stopping with a smaller available budget consistently harms task rating compared with the original PACT setting, indicating that abrupt budget exhaustion can interrupt useful agent coordination before completion. This effect is especially severe in MARBLE, whose star topology makes task progress tightly coupled to continued interaction among agents. Average task rating drops by 40% when the budget is capped at average task cost and 77% when the budget is further reduced to 75% of average cost.

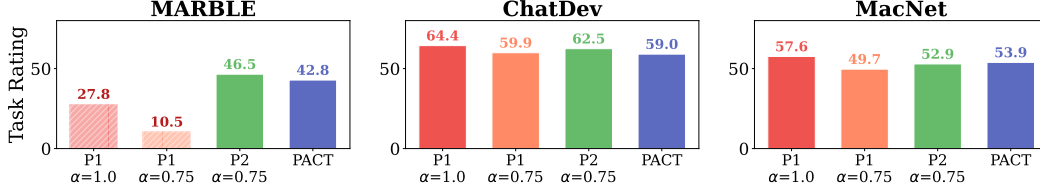


Figure 7: Budget-enforcement policies vs. PACT ($n = 30$, DeepSeek-V3.2). Hatched bars indicate incomplete tasks. P1 (Hard Stop) causes severe degradation in MARBLE but not in ChatDev or MacNet; P2 (Active Approve) recovers quality to PACT levels across all frameworks.

(2) When the available balance is larger, or when the supervisor actively approves continuation, task performance recovers to, or even exceeds, the original PACT level across all frameworks. This suggests that budget awareness can help the system allocate computation more effectively and avoid unproductive spending. Full numerical results are provided in Table 5 (Appendix C).

5.4 Multi-provider Scalability

In production, agents may invoke different LLM providers, requiring the payment layer to maintain multiple payment channels concurrently. Using MARBLE Coding with DeepSeek-V3.2, we vary the number of active payment channels $P \in \{1, 2, 4\}$ by assigning agents to virtual providers via round-robin, with $P=1$ corresponding to the single-channel PACT baseline. Figure 8 plots the task rating and the latency overhead as the number of providers varies from 1 to 4.

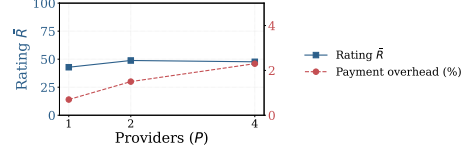


Figure 8: Multi-channel scalability ($n=30$, DeepSeek-V3.2).

We have two main conclusions: (1) task quality remains stable as the number of channels increases, indicating that multi-provider payment coordination does not interfere with MAS execution.

(2) Payment overhead grows approximately linearly with the number of channels, as each additional provider introduces its own channel open/close transactions. Even with four channels, however, the total payment overhead remains below 2.5%, showing that PACT scales to multi-provider settings while preserving negligible runtime overhead.

6 Conclusion

This paper provides, to our knowledge, the first systematic treatment of the MAS payment problem. Empirical profiling reveals three defining properties of MAS workloads—high-frequency low-value transactions, near-linear cost accumulation, and heterogeneous per-agent cost distribution—that are jointly incompatible with both API-key billing and direct on-chain settlement. Motivated by these properties, we present PACT, an agent-centric three-layer architecture that forms an economic substrate for MAS. Across multiple MAS frameworks and backbone models, PACT preserves task quality while introducing negligible cost and latency overhead and eliminating settlement failures compared with direct on-chain settlement. More broadly, this work highlights a growing mismatch between agentic workloads and infrastructure inherited from human-centric ecosystems. For instance, mechanisms such as API keys were designed around human accounts; when directly applied to autonomous multi-agent execution, they become rigid grafts rather than native foundations. We hope this work provides one step toward such infrastructure foundations for the agent era.

Limitations and future work. PACT currently uses static, task-level budgets and treats all payment channels uniformly, without adapting to per-agent cost patterns or dynamically reallocating funds during execution. Extending PACT with per-agent dynamic budgets and behavior-aware channel management are promising directions. More broadly, by lowering the barrier for autonomous agents to transact independently, systems like PACT may reduce human oversight of agent economic activity, underscoring the need for complementary compliance and auditing mechanisms.

References

- [1] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. Chatdev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [2] Xingyao Wang et al. Openhands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.
- [3] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [4] Zhonghan Zhao et al. STEVE series: Step-by-step construction of agent systems in Minecraft. *arXiv preprint arXiv:2406.11247*, 2024.
- [5] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.
- [6] Haoifei Yu et al. Researchtown: Simulator of human research community. *arXiv preprint arXiv:2412.17767*, 2025.
- [7] Shuang Xu et al. Werewolf arena: A case study in LLM evaluation via social deduction. *arXiv preprint arXiv:2407.13943*, 2025.
- [8] Joseph Poon and Thaddeus Dryja. The Bitcoin lightning network: Scalable off-chain instant payments. 2016. White paper.
- [9] Mo Dong, Qingkai Zhong, Chenxing Li, Junda Liang, et al. Celer network: Bring internet scale to every blockchain. 2018. White paper, <https://celer.network>.
- [10] Kunlun Zhu et al. MultiAgentBench: Evaluating the collaboration and competition of LLM agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.
- [11] Chen Qian, Zihao Xie, Yifei Wang, Wei Liu, Yufan Dang, Zhuoyun Du, Weize Chen, Cheng Yang, Xin Cong, Xiaowei Che, et al. Scaling large language model-based multi-agent collaboration. *arXiv preprint arXiv:2406.07155*, 2024.
- [12] Sirui Hong et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024. Oral presentation.
- [13] Wei Tao, Yucheng Zhou, Yanlin Wang, Wenqiang Zhang, Hongyu Zhang, and Yu Cheng. MAGIS: LLM-based multi-agent framework for github issue resolution. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [14] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xiong-Hui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. AFlow: Automating agentic workflow generation. In *International Conference on Learning Representations (ICLR)*, 2025. Oral presentation.
- [15] Khanh-Tung Tran, Dung Dao, Minh-Duong Nguyen, Quoc-Viet Pham, Barry O’Sullivan, and Hoang D. Nguyen. Multi-agent collaboration mechanisms: A survey of LLMs, 2025.
- [16] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen LLM applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024.
- [17] LangChain, Inc. Langgraph: Agent runtime and orchestration framework. <https://github.com/langchain-ai/langgraph>, 2024.

- [18] OpenAI. Swarm: An educational multi-agent orchestration framework. <https://github.com/openai/swarm>, 2024.
- [19] MCP Manager. Model context protocol adoption statistics. <https://mcpmanager.ai/blog/mcp-adoption-statistics/>, 2025.
- [20] Google. A2A: A new era of agent interoperability. <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/>, 2025.
- [21] Boye Niu, Yiliao Song, Kai Lian, Yifan Shen, Yu Yao, Kun Zhang, and Tongliang Liu. Flow: Modularized agentic workflow automation. In *International Conference on Learning Representations (ICLR)*, 2025.
- [22] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. <https://bitcoin.org/bitcoin.pdf>, 2008.
- [23] Stefan Dziembowski, Sebastian Faust, and Kristina Hostáková. General state channel networks. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 949–966, 2018.
- [24] Bardia Vaziry et al. Towards trustless machine-to-machine economies: An A2A-compatible payment framework using DLT-based AgentCards and x402 micropayments. *arXiv preprint arXiv:2507.19550*, 2025.
- [25] Nevermined AG. Nevermined: Payments infrastructure for AI agents. <https://nevermined.io>, 2025. Accessed: 2026-04-25.
- [26] Ben Goertzel, Simone Giacomelli, David Hanson, Cassio Pennachin, and Marco Argentieri. Singularitynet: A decentralized, open market and network for AI services. *arXiv preprint arXiv:1911.00622*, 2019.
- [27] Helicone Inc. Helicone: Open-source LLM observability platform. <https://helicone.ai>, 2024. Accessed: 2026-04-25.
- [28] LangChain Inc. Langsmith: LLM application tracing and evaluation. <https://smith.langchain.com>, 2024. Accessed: 2026-04-25.
- [29] Lunary. Lunary: Open-source LLM observability. <https://lunary.ai>, 2024. Accessed: 2026-04-25.
- [30] Etherscan. Ethereum gas tracker. <https://etherscan.io/gastracker>, 2026.
- [31] L2BEAT. L2BEAT – layer 2 transaction costs and finality. <https://l2beat.com>, 2025.
- [32] DeepSeek-AI. Deepseek-v3.2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.
- [33] OpenAI. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.
- [34] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [35] BerriAI. Litellm: A unified interface for LLM providers. <https://github.com/BerriAI/litellm>, 2024. Accessed: 2026-04-28.
- [36] Gavin Wood. Ethereum: A secure decentralised generalised transaction ledger. <https://ethereum.github.io/yellowpaper/paper.pdf>, 2014. Yellow paper, revised 2024.
- [37] Coinbase. Introducing Base. <https://base.org>, 2023.
- [38] Anatoly Yakovenko. Solana: A new architecture for a high performance blockchain. <https://solana.com/solana-whitepaper.pdf>, 2018.

A Unified Report Construction

Algorithm 1 details how the unified status report is constructed from middleware-produced fields (per-agent cost, budget state) and framework-extracted fields (task progress, agent history, memory) obtained through the plugin adapter described in §4.2. The supervisor consumes this report at each balance signal to make budget-enforcement decisions.

Algorithm 1 Unified status report construction.

Require: MAS framework state S , middleware cost ledger L , plugin adapter A
Ensure: Structured report R for the supervisor

— **Middleware-produced (universal)** —

- 1: **for** each agent a **do**
- 2: $R.\text{cost}[a] \leftarrow L.\text{aggregate}(a)$ ▷ total cost, call count, tokens
- 3: **end for**
- 4: $R.\text{budget} \leftarrow (C_{\text{total}}, B, C_{\text{unpaid}})$

— **Framework-extracted (via adapter A)** —

- 5: $R.\text{task} \leftarrow A.\text{GETTASK}(S)$ ▷ task objective
- 6: $R.\text{topology} \leftarrow A.\text{GETMODE}(S)$ ▷ star|chain|dag
- 7: $R.\text{progress} \leftarrow A.\text{GETPROGRESS}(S)$
- 8: **for** each agent a **do**
- 9: $R.\text{agents}[a].\text{identity} \leftarrow A.\text{GETPROFILE}(a)$
- 10: $R.\text{agents}[a].\text{history} \leftarrow \text{TRUNC}(A.\text{GETHISTORY}(a))$
- 11: $R.\text{agents}[a].\text{memory} \leftarrow \text{TRUNC}(A.\text{GETMEMORY}(a))$
- 12: **end for**
- 13: **return** R

B Payment Workload Details

This appendix provides the full quantitative data underlying the payment workload characterization in §3. All MAS data are drawn from the 6,589-call PACT corpus; blockchain statistics are sourced from public dashboards [30, 31] as of early 2026.

B.1 Per-Call Cost Statistics

Table 3 summarizes the per-call cost distribution across the PACT corpus.

Table 3: Per-call cost statistics across the PACT corpus (6,589 calls).

Framework	# calls	mean (\$)	median (\$)	std (\$)	p95 (\$)	max (\$)
ChatDev	1,261	0.001009	0.000486	0.0011	0.003502	0.005
MacNet	1,446	0.000894	0.000447	0.0010	0.003039	0.004
MARBLE	3,882	0.001895	0.001720	0.0014	0.004877	0.010
All	6,589	0.001506	0.001168	0.0014	0.004072	0.010

B.2 Blockchain Fee Comparison

Table 4 compares the median per-call cost of each MAS framework against the average transaction fee on eight blockchains. The fee-to-payment ratio (rightmost columns) expresses the blockchain fee as a percentage of the framework’s median call cost; values above 100% indicate that the fee exceeds the payment.

Table 4: Blockchain transaction fees vs. median per-call MAS cost. Fee-to-payment ratio $>100\%$ (bold) means the fee exceeds the payment value. *Finality* refers to soft finality (see Table 2 caption for definition).

Chain	Avg fee (USD)	Finality	Fail rate	Fee / median call cost (%)		
				ChatDev	MacNet	MARBLE
Ethereum L1	\$0.34	~ 13 min	$\sim 1\%$	70,005%	76,066%	19,765%
BNB Chain	\$0.03	~ 3 s	$<1\%$	6,177%	6,712%	1,744%
Arbitrum	\$0.20	~ 1 s	$<1\%$	41,179%	44,745%	11,627%
Optimism	\$0.15	~ 1 s	$<1\%$	30,885%	33,559%	8,720%
Base	\$0.01	~ 1 s	$<1\%$	2,059%	2,237%	581%
Polygon	\$0.004	~ 2 min	$<1\%$	824%	895%	233%
Avalanche	\$0.02	<1 s	$<1\%$	4,118%	4,474%	1,163%
Solana	\$0.00025	~ 13 s	6–73%	51%	56%	15%

C Budget-Enforcement Detailed Results

This appendix provides the full numerical results for the budget-enforcement experiments described in §5.3.

C.1 DeepSeek-V3.2

Table 5 reports task rating ($\bar{R}$), completion count, average cost ($\bar{C}$), average wall-clock time ($\bar{T}$), and payment-channel overhead percentage for each framework under the API-Key baseline, PACT (unlimited budget), P1 (Hard Stop) with $\alpha \in \{0.75, 1.00\}$, and P2 (Active Approve) with $\alpha = 0.75$.

Table 5: Budget-enforcement policies vs. API-Key and PACT ($n = 30$, DeepSeek-V3.2).

Framework	Condition	$\bar{R}$	Compl.	$\bar{C}$ (\$)	$\bar{T}$ (s)	Pay. %
MARBLE	API-Key	47.50	30/30	—	1073	—
	PACT	42.83	30/30	0.079	846	0.7%
	P1 $\alpha=0.75$	10.50	7/30	0.054	784	1.3%
	P1 $\alpha=1.00$	27.83	17/30	0.056	654	1.0%
	P2 $\alpha=0.75$	46.50	30/30	0.091	1224	0.5%
ChatDev	API-Key	63.48	30/30	—	763	—
	PACT	59.04	30/30	0.032	768	0.9%
	P1 $\alpha=0.75$	59.92	30/30	0.025	700	0.8%
	P1 $\alpha=1.00$	64.36	30/30	0.027	626	1.1%
	P2 $\alpha=0.75$	62.45	30/30	0.030	829	0.4%
MacNet	API-Key	56.08	30/30	—	1119	—
	PACT	53.86	30/30	0.032	963	0.8%
	P1 $\alpha=0.75$	49.66	30/30	0.024	963	0.7%
	P1 $\alpha=1.00$	57.60	30/30	0.028	946	0.8%
	P2 $\alpha=0.75$	52.90	30/30	0.031	1143	0.6%

C.2 Qwen3-32B

Table 6 extends the budget-enforcement analysis to Qwen3-32B.

D AFlow: Supplementary Framework Validation

To further validate the generality of PACT beyond the three primary frameworks, we conduct supplementary experiments with AFlow [?], a workflow optimization framework that uses Monte Carlo Tree Search (MCTS) to iteratively improve multi-agent workflows. Unlike the primary frameworks

Table 6: Budget-enforcement policies on ChatDev with Qwen3-32B ($n = 30$).

Framework	Condition	$\bar{R}$	Compl.	$\bar{C}$ (\$)	$\bar{T}$ (s)	Pay. %
ChatDev	API-Key	43.94	30/30	—	450	—
	PACT	51.72	30/30	0.006	479	0.6%
	P1 $\alpha=0.75$	42.66	30/30	0.004	325	1.0%
	P1 $\alpha=1.00$	45.81	30/30	0.005	359	1.4%
	P2 $\alpha=0.75$	45.33	30/30	0.005	452	1.2%

(star, chain, DAG topologies), AFlow represents a *meta-optimization* setting: an optimizer agent (Claude-Sonnet-4.6) proposes candidate workflow graphs, and an executor model evaluates each candidate on a validation set; MCTS guides the search over up to 10 rounds. We evaluate on GSM8K (Table 7, three executor models) and MATH (Table 8, two executor models; GPT-OSS-120B is omitted due to incomplete runs). Across all configurations, PACT preserves optimization accuracy while introducing payment-channel overhead below 1%, consistent with the findings on the primary frameworks (§5.2).

Table 7: AFlow on GSM8K: API-Key vs. PACT. Accuracy is the best validation accuracy across all optimization rounds. Pay. % is the fraction of wall-clock time spent on payment-channel operations.

Executor model	Condition	Accuracy (%)	Pay. %
DeepSeek-V3.2	API-Key	95.83	—
	PACT	96.36	0.3%
Qwen3-32B	API-Key	96.89	—
	PACT	96.89	0.9%
GPT-OSS-120B	API-Key	97.65	—
	PACT	97.27	0.2%

Table 8: AFlow on MATH: API-Key vs. PACT. GPT-OSS-120B is omitted due to incomplete runs.

Executor model	Condition	Accuracy (%)	$\bar{T}$ (s)	Pay. (s)	Pay. %
DeepSeek-V3.2	API-Key	85.55	66,815	—	—
	PACT	86.39	26,166	70	0.3%
Qwen3-32B	API-Key	75.63	21,757	—	—
	PACT	75.97	29,310	74	0.3%